
Performance Analysis and Optimization of MiniCFD on a Shared-Memory Node

Anonymous Submission



TECHNISCHE
UNIVERSITÄT
DARMSTADT

Abstract

This paper studies the OpenMP performance of MiniCFD on a single node of the Lichtenberg cluster. The focus is the pressure-correction phase, which dominates the runtime of the application and limits parallel scaling. A reproducible benchmark campaign was built around an output-free run configuration, automated Slurm execution, repeated measurements, and HPCToolkit profiling. The baseline configuration with the default DIC preconditioner showed very weak scaling: runtime improved only from 136.22 s at one thread to 108.52 s at 16 threads, and higher thread counts did not help. Profiling identified two main issues: expensive pressure-solver work inside preconditioned conjugate gradient (PCG), and unnecessary temporary-vector overhead in the solver loop. The implemented optimizations were a reduction of shared grid-access overhead in the advection phase, in-place vector updates in PCG, and reuse of already computed solver quantities. In the final version, the DIC configuration improved to 81.66 s at 16 threads and 82.30 s at 104 threads. A comparative Jacobi campaign, starting from the common Opt-A code state, showed that the same solver-side changes reduced the 8-thread runtime further to 80.00 s. The remaining bottleneck is the limited scalability of the pressure solver, especially the sequential DIC preconditioner and the memory-bound sparse matrix-vector product.

1 Introduction

MiniCFD is a compact teaching application for incompressible flow simulation. Its main value in a performance-engineering course is not peak realism, but the fact that it contains a complete numerical workflow in a code base that is small enough to inspect directly. That makes it a good target for a single-node study: timing can be automated, hotspots can be inspected in detail, and code changes can be related back to concrete numerical kernels.

The goal of this work is to answer a focused question: what limits the performance of MiniCFD on one shared-memory node, and which small, justified optimizations can improve it? The scope is deliberately restricted to OpenMP on one Lichtenberg node. No MPI scaling study, compiler comparison, or multi-node analysis is included.

The study follows a standard performance-engineering loop. First, a stable baseline is established. Second, hotspot analysis is used to identify the dominant costs. Third, small code changes are introduced one at a time and re-measured under the same benchmark conditions. This approach keeps causality clear: every result can be tied back to one specific change or one specific design decision.

2 Related Work and Tooling

The numerical core of MiniCFD is a standard sparse iterative-solver problem. Preconditioned conjugate gradient and related methods are classical tools for large sparse linear systems. Saad [1, pp. 107, 113] discusses how classical

iterations such as Jacobi and Gauss-Seidel can be interpreted through preconditioning, which is exactly the solver setting relevant for MiniCFD. From that perspective, the MiniCFD pressure solver is representative: it combines a sparse matrix-vector product, repeated dot products, and preconditioner application inside a tight convergence loop.

For performance analysis, Adhianto et al. [2, p. 685] describe HPCToolkit as an integrated suite for measurement, analysis, attribution, and presentation of performance data for sequential and parallel programs. They also motivate measurement without intrusive instrumentation for optimized code [2, p. 686]. OpenMP behavior was interpreted together with source inspection instead of relying on runtime metrics alone. This is important for MiniCFD because some limits are not caused by a bad OpenMP setting, but by the algorithmic structure of the underlying kernels.

3 Application Background

MiniCFD evolves a velocity field on a uniform three-dimensional Cartesian grid. For the experiments in this paper, the command-line parameters have the following meaning:

- `-d`: number of grid cells per spatial axis,
- `-e`: simulation end time,
- `-s`: time-step size,
- `-p`: preconditioner choice for the pressure solver.

For a domain size of $d = 96$, the simulation grid contains $96^3 = 884,736$ cells. With `-e 10` and `-s 0.4`, the application performs $10/0.4 = 25$ time steps. Each time step consists mainly of three phases:

1. semi-Lagrangian advection of the velocity field,
2. pressure correction by solving a sparse linear system,
3. application of the pressure correction to enforce incompressibility.

The pressure correction is the most expensive phase. In the current code, the pressure matrix is assembled from a 7-point Laplace stencil in every time step. The resulting sparse system is solved with a preconditioned conjugate gradient (PCG) method. Two preconditioners were studied:

- Jacobi, which is simple and cheap per iteration,
- DIC, a simplified diagonal-based incomplete Cholesky variant.

In this code base, this creates a practical tradeoff. A stronger preconditioner can reduce the number of PCG iterations, but each preconditioner application is more expensive. A weaker preconditioner is cheaper to apply, but it can leave more work to the Krylov solver and therefore increase the total cost of sparse matrix-vector products and reductions over the whole iteration process.

This structure already suggests where bottlenecks may appear. In MiniCFD, the sparse matrix-vector product performs indirect accesses through the sparse matrix structure and repeatedly gathers values from the input vector. The DIC implementation also performs forward and backward sweeps over dependent entries. Both patterns restrict scalable parallel work, which is consistent with the observed profiles and scaling curves.

4 Experimental Setup

4.1 Platform and Build

All main measurements were performed on the Lichtenberg cluster using the course reservation `kurs00102`. Jobs were executed through Slurm on a single node with up to 104 hardware threads. The code was compiled with GCC 11.5.0. The main benchmark build used `RelWithDebInfo` with `-O2 -g -DNDEBUG`. Profiling tools were provided through the prepared Spack environment.

The work used two storage locations according to the cluster policy. The Git repository, scripts, and derived summaries stayed in the persistent home directory. Larger raw outputs, including per-run logs and HPCToolkit data, were written to `$HPC_SCRATCH`.

4.2 Benchmark Design

Benchmark automation was added to the repository in the form of build scripts, campaign generators, Slurm wrappers, result collectors, and CSV aggregators. For performance measurements, a new command-line flag `-disable-output` was introduced so that file I/O did not distort the compute-oriented benchmarks.

The pilot campaign compared domain sizes 64, 96, and 128 with the DIC preconditioner. All runs used `-s 0.4`, disabled output, and three repetitions. The results are shown in Table 1. Domain size 64 was already measurable, but 96 was chosen as the main benchmark because it offered a better compromise between runtime, profiling visibility, and cluster queue constraints. Domain size 128 was unnecessarily expensive for repeated experiments.

Table 1: Pilot measurements used to choose a representative problem size.

Domain size	Threads	Mean runtime [s]	16-thread speedup
64	1	30.48	–
64	16	22.37	1.35
96	1	135.92	–
96	16	108.74	1.25
128	1	537.57	–
128	16	464.70	1.16

The final benchmark setup used:

- domain size `-d 96`,
- end time `-e 10`,
- time-step size `-s 0.4`,
- disabled output,
- three repetitions per configuration.

The DIC campaigns form the full baseline-to-final optimization chain. The Jacobi campaigns were added later, after Optimization A was already present in the working tree. As a result, the Jacobi progression reported in this paper starts from the common Opt-A code state rather than from a pre-opt-a baseline.

An OpenMP placement calibration compared `OMP_PROC_BIND=close` and `spread` at 16, 32, 52, and 104 threads. The difference was negligible: for example, the 16-thread DIC runtime changed only from 108.04s (`close`) to 108.72s (`spread`). Since the performance was effectively identical, `spread` was kept for the remaining experiments.

4.3 Measurement Quality

Relative standard deviations in the main timing campaign stayed below one percent in all reported DIC runs and below half a percent in the main Jacobi runs. For example, the baseline DIC case ranged from 0.16% to 0.71%. This indicates that the benchmark setup was stable enough for comparing small code changes.

4.4 Automation and Reproducibility

An important part of the project was not only the measurements themselves, but the ability to repeat them. The repository was extended with a small automation layer for cluster builds, campaign generation, Slurm execution, result collection, aggregation, and plotting. Every run produced machine-readable metadata, including the Git commit, build name, compiler flags, node name, thread count, OpenMP placement, simulation parameters, exit code, and wall time. This made it possible to compare versions without relying on manually copied command lines or ad hoc notes.

This reproducibility work mattered in practice. Several optimization ideas were tested in close succession, including one intentionally unsuccessful change. Without automated campaign generation and aggregated CSV summaries, it would have been very difficult to separate real improvements from random variation or configuration mistakes. The measurement pipeline therefore became part of the engineering result, not just a convenience script around it.

5 Baseline Characterization

Table 2 shows the baseline scaling of the default DIC configuration. The most important result is not the absolute runtime, but the shape of the scaling curve. Performance improves somewhat up to 8–16 threads, then essentially stops improving. At 104 threads, the runtime is even slightly worse than at 16 threads.

Table 2: Baseline scaling with the default DIC preconditioner.

Threads	Mean runtime [s]	Speedup	Efficiency
1	136.22	1.00	1.000
2	124.56	1.09	0.544
4	113.53	1.20	0.299
8	109.68	1.24	0.155
16	108.52	1.25	0.078
32	109.25	1.24	0.039
52	109.31	1.24	0.024
64	109.52	1.24	0.019
104	112.81	1.20	0.012

This baseline already reveals the core problem: MiniCFD uses OpenMP, but the pressure-solver path does not expose enough scalable work to benefit from very high thread counts. A pure placement change cannot solve that. The reason must be inside the numerical kernels themselves.

6 Optimizations

Unless stated otherwise, the optimization stages are cumulative: Opt-C builds on Opt-A, and Opt-D builds on Opt-C. The only exception is Opt-B, which was an isolated Jacobi experiment and was not carried forward.

6.1 Optimization A: Remove Hot-Path Grid-Access Overhead

The first optimization was motivated by the baseline DIC profiles. The serial run showed that `solveAdvection(...)` still consumed 13.4% of total runtime, which was too large to ignore early. The 16-thread worker-thread view then showed a much more specific pathology: 50.2% of worker-thread time was spent inside `semiLagrangianAdvection(...)`, and 24.2% was spent in `gomp_simple_barrier_wait`. Drilling into this region identified `traceBackward(...)` and `VelocityField::trilerp(...)` as the hot subroutines.

The corresponding source-level issue was small but highly repeated. The advection code accessed the grid through a `shared_ptr` interface in a hot OpenMP region. A const grid-reference accessor was added and used in the advection path, so that the numerical method remained unchanged while pointer and reference-management overhead were removed from the hottest inner loop.

The performance effect was modest in serial but visible in parallel. For the default DIC configuration, the 16-thread runtime improved from 108.52s to 103.69s, while the one-thread runtime changed only from 136.22s to 134.92s. This was exactly the expected pattern for a parallel hot-path cleanup. Table 3 summarizes the effect at representative thread counts.

Table 3: Effect of Optimization A on the DIC configuration.

Threads	Baseline [s]	Opt-A [s]	Change [%]
1	136.22	134.92	-1.0
8	109.68	104.78	-4.5
16	108.52	103.69	-4.4
104	112.81	105.32	-6.6

6.2 Optimization C: Replace Temporary-Heavy PCG Vector Expressions

After Optimization A, the advection phase became less dominant and the next profile clearly pointed at the solver. In the post-Opt-A DIC profile, the main costs were concentrated in three places: the PCG driver, the

DIC preconditioner, and the sparse matrix-vector product. The same profile also showed repeated cost in vector operators such as addition, scalar multiplication, and assignment.

The serial Opt-A profiles were especially useful because they separated generic solver overhead from preconditioner-specific cost. In the DIC Opt-A serial run, `DICPreconditioner::precondition(...)` still dominated with 34.0%, `spmv(...)` contributed 16.0%, and `solveAdvection(...)` remained at 13.5%. In the Jacobi Opt-A serial run, the balance shifted strongly toward generic PCG work: `spmv(...)` rose to 29.8%, the Jacobi preconditioner itself accounted for only 4.8%, and several overloaded vector operators inside the same PCG loop each contributed another 4–8%. The key conclusion was that the next useful change should target the shared PCG update loop rather than one specific preconditioner implementation.

The relevant code pattern inside PCG was

$$x = x + \alpha d, \quad r = r - \alpha z, \quad d = h + \beta d.$$

One PCG iteration therefore performs one sparse matrix-vector product, several dot products, one preconditioner application, and then these three vector updates. HPCToolkit showed that this loop body was executed often enough that apparently “small” operator-overload costs accumulated into a visible fraction of total runtime.

This style is compact, but each expression may allocate or copy temporary vectors. Since the PCG loop executes many times per time step, even moderate temporary-creation overhead becomes expensive in total.

The change was therefore to replace the temporary-heavy expressions with in-place vector helpers: one AXPY-style update for the solution, one AXPY-style update for the residual, and one combined update for the search direction. This did not alter the solver mathematics. It only reduced intermediate objects and memory traffic.

This was the largest single optimization of the project. For DIC, the 16-thread runtime improved from 103.69 s to 84.87 s. For Jacobi, which was measured from the same Opt-A code state, the 16-thread runtime improved from 163.66 s to 97.37 s. The stronger effect in Jacobi is consistent with the HPCToolkit profile: after Opt-C, Jacobi spent about 43.1% of its time in `spmv(...)` and 16.5% in dot products, while `JacobiPreconditioner::precondition(...)` itself contributed only 7.9%. In other words, Jacobi exposed more of the generic PCG machinery, so reducing vector overhead helped more. Table 4 compares the preconditioners on matching thread counts before and after this solver rewrite.

Table 4: Opt-A to Opt-C progression on matching thread counts for both preconditioners.

Threads	DIC Opt-A [s]	DIC Opt-C [s]	Jacobi Opt-A [s]	Jacobi Opt-C [s]
1	134.92	113.03	223.67	153.46
8	104.78	86.06	165.37	99.33
16	103.69	84.87	163.66	97.37
104	105.32	84.88	169.94	106.10

6.3 Optimization D: Reuse Existing Solver Quantities

After Optimization C, the remaining solver overhead was smaller, but still visible. The next low-risk target was repeated work inside the same PCG loop. Two opportunities were used.

First, the step size originally recomputed an inner product that was already available as `oldSqrResidNorm`. Second, Jacobi preconditioning still returned a fresh vector on every application, even though the solver could reuse a preallocated output vector. Both issues are small in isolation, but they occur in the hottest iterative region of the program.

The change was therefore to reuse the existing scalar `oldSqrResidNorm` in the step-size calculation and to add an in-place preconditioner interface for Jacobi. This again preserved the numerical method. It only removed repeated work and repeated object creation.

The result was a smaller but still measurable improvement. For DIC, the 16-thread runtime improved from 84.87 s to 81.66 s. For Jacobi, the 16-thread runtime improved from 97.37 s to 81.00 s, and the 8-thread runtime improved from 99.33 s to 79.97 s. This confirmed that the solver still contained profitable low-level inefficiencies even after the larger Opt-C rewrite.

6.4 Optimization B: Failed OpenMP Experiment on Jacobi

One negative result is worth keeping because it clarifies what did not work. A separate experiment parallelized the Jacobi preconditioner itself with a simple OpenMP loop. This looked attractive because Jacobi is structurally simpler than DIC, but the measurement contradicted that intuition. At 8 threads, the Jacobi runtime worsened from 165.37s to 167.66s, and at 104 threads it worsened from 169.94s to 178.66s.

This failed optimization supports the main argument of the paper: the project was not improved by adding directives blindly. Improvements came from following the measured hotspot sequence. In this case, the full PCG path mattered more than a naive OpenMP change to one small component.

6.5 Performance Summary

The cumulative DIC progression is shown in Table 5. Optimization A is mainly a parallel cleanup. Optimization C delivers the main solver improvement. Optimization D adds a smaller final refinement.

Table 5: Runtime progression for the default DIC configuration.

Version	1 thread [s]	16 threads [s]	104 threads [s]
Baseline	136.22	108.52	112.81
Opt-A	134.92	103.69	105.32
Opt-C	113.03	84.87	84.88
Opt-D	110.70	81.66	82.30

Compared to baseline, the final DIC version reduced runtime by about 18.7% at one thread, 24.8% at 16 threads, and 27.0% at 104 threads. The absolute improvement is therefore real and substantial. However, the scaling curve is still unsatisfactory. Even in the final version, the speedup at 16 threads is only 1.35, and performance stays almost flat between 16 and 104 threads.

Table 6: Final Opt-D comparison between the two preconditioners at matching thread counts.

Threads	DIC [s]	Jacobi [s]
1	110.70	135.06
2	97.63	103.25
4	88.42	86.64
8	83.93	79.97
16	81.66	81.00
32	81.78	87.69
52	80.93	87.24
104	82.30	92.03

The final comparison in Table 6 is instructive because both columns come from the same final Opt-D code state and therefore differ only in the selected preconditioner. DIC is clearly better in serial execution because its stronger preconditioner reduces solver work more effectively. Jacobi, however, becomes competitive once the solver-side overheads are reduced. At 8 and 16 threads, the two methods reach almost identical runtimes, even though they arrive there for different reasons: DIC spends more time in preconditioning, while Jacobi shifts more work toward `spmv()` and reductions. This supports the main profiling conclusion that MiniCFD is limited less by one universal bottleneck and more by the interaction between solver structure and memory behavior.

The same table also makes the scaling limit visible in a way that a single curve does not. Both preconditioners improve clearly from 1 to 8 threads. Between 8 and 16 threads, the gains become small. Beyond that point the curves either flatten completely (DIC) or turn upward again (Jacobi). This is the measured consequence of the hotspot analysis: once the code is dominated by memory-bound `spmv()`, reductions, and in the DIC case additionally by sequential triangular sweeps, adding more OpenMP threads no longer creates proportional useful work.

6.6 Why Scaling Still Flattens

The final code is clearly faster than the baseline, but it still does not scale like a highly optimized production solver. There are several reasons:

- The DIC preconditioner contains sequential forward and backward sweeps.
- The sparse matrix-vector product uses indirect memory accesses and is therefore bandwidth-sensitive.
- The PCG loop is mathematically sequential across iterations. Only the vector kernels inside each iteration can be parallelized.

These points also explain why thread counts above 8 or 16 bring little benefit. The application does not have enough scalable work per iteration to keep 32, 52, or 104 threads productive. In the profiles, this appears as substantial OpenMP waiting time.

7 Discussion

The most useful lesson from this study is that performance improvements did not come from one large algorithmic rewrite. They came from identifying where the program actually spends time and then removing overheads in the hottest path.

The strongest implemented optimization was not a new numerical method. It was a change in data movement: replacing temporary-heavy vector expressions with in-place updates. Even when the floating-point formulas stay identical, the cost of allocation, copying, and extra passes over memory can dominate a large fraction of runtime.

The second lesson is methodological. The failed Jacobi OpenMP experiment was not wasted work. It ruled out an intuitive but incorrect optimization strategy and made the later improvements more defensible. In a performance paper, negative results are useful when they narrow the search space and support the final argument.

Finally, the profiles show that MiniCFD still contains further optimization potential. The most important remaining limitation is the pressure solver itself. Beyond the implemented low-level cleanups, the code would benefit from a more parallel-friendly solver and preconditioner combination than the current PCG plus DIC path.

8 Conclusion

This paper presented a single-node OpenMP performance study of MiniCFD on Lichtenberg. The baseline application showed weak scaling with the default DIC preconditioner: 16 threads improved runtime only from 136.22 s to 108.52 s. HPCToolkit identified the pressure-correction phase as the main bottleneck, with different dominant kernels for DIC and Jacobi.

Three useful optimization directions were validated. First, repeated hot-path grid access in advection was reduced. Second, the PCG loop was rewritten to use in-place vector updates instead of temporary-heavy expressions. Third, already computed solver quantities were reused and Jacobi preconditioning was converted to an in-place form. Together, these changes reduced the DIC runtime to 81.66 s at 16 threads and the Jacobi runtime to 79.97 s at 8 threads.

The remaining limitation is not measurement noise or poor job placement. It is algorithmic and structural: DIC contains sequential sweeps, `spmv()` is memory-bound, and the PCG loop exposes only limited thread-parallel work per iteration.

9 Future Work

The most relevant future work is not more micro-tuning of OpenMP settings, but reconsidering the pressure-solver design itself.

The pressure solver would benefit from a more parallel-friendly algorithmic combination, for example a different preconditioner or a solver variant that exposes more useful work inside each iteration.

Disclosure

OpenAI Codex was used for wording refinement and L^AT_EX editing support. Benchmark data, profiling runs, code execution, interpretation of results, and the final judgment are the responsibility of the author.

References

- [1] Y. Saad. Iterative Methods for Sparse Linear Systems. Society for Industrial and Applied Mathematics, 2nd edition, 2003.
- [2] S. Adhianto, S. Banerjee, M. Fagan, M. Kruskal, Q. Liu, L. Yuan, and W. N. Scherer. HPCToolkit: Tools for performance analysis of optimized parallel programs. *Concurrency and Computation: Practice and Experience*, 22(6):685–701, 2010.